

机器无关代码优化
Part4：数据流与活跃变量分析

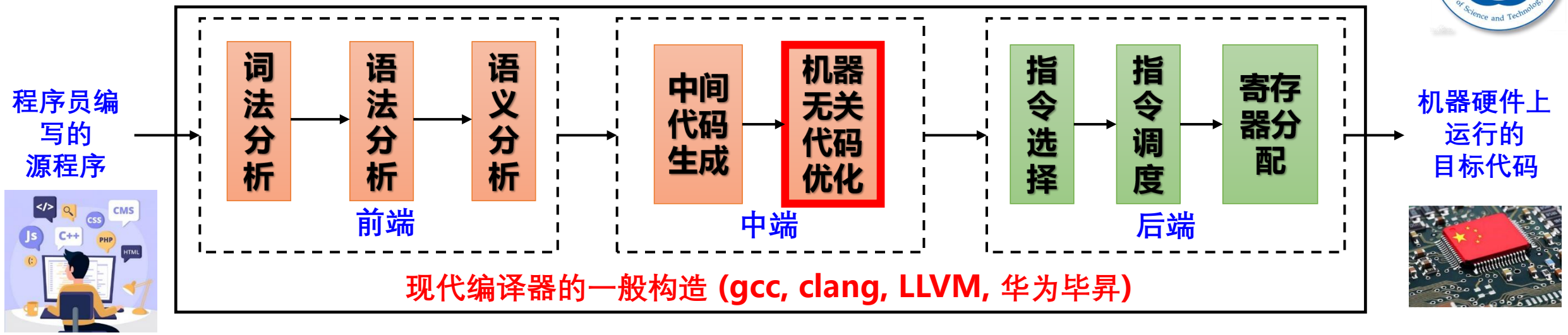
李 诚

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2026年11月9日

本节提纲



- 活跃变量定义及应用
- 活跃变量分析算法
- 示例驱动的分析流程

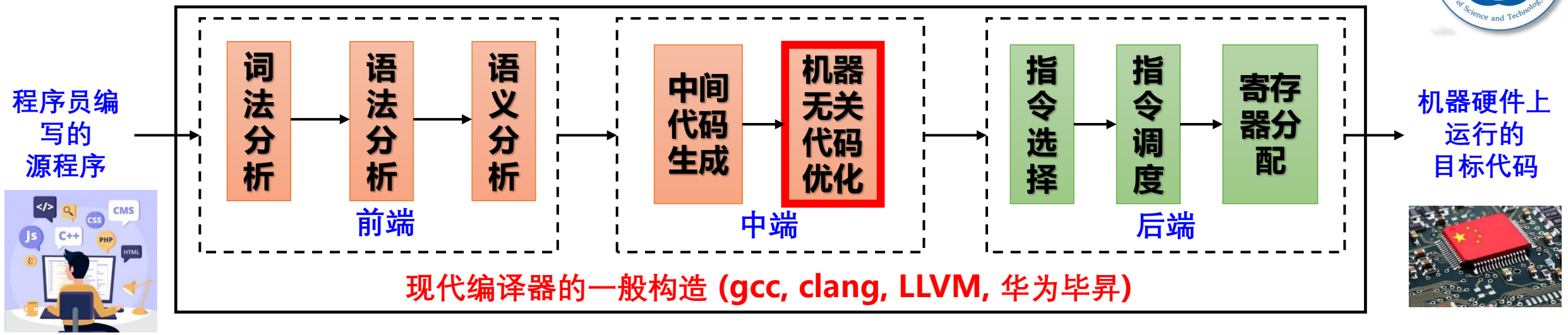
活跃变量

- **定义：**
  - 对于变量x和程序点p，如果x的值在*p*点开始的某条执行路径上被引用，则说x在*p*点活跃（live），否则称x在*p*点已经死亡（dead）

活跃变量分析的应用

- **为基本块分配寄存器**
  - 如果所有寄存器都被占用，且还需要申请一个寄存器，则应该考虑使用已经存放死亡值的寄存器
  - 如果一个值在基本块结尾处是死的，就不必在结尾处保存这个值了

本节提纲



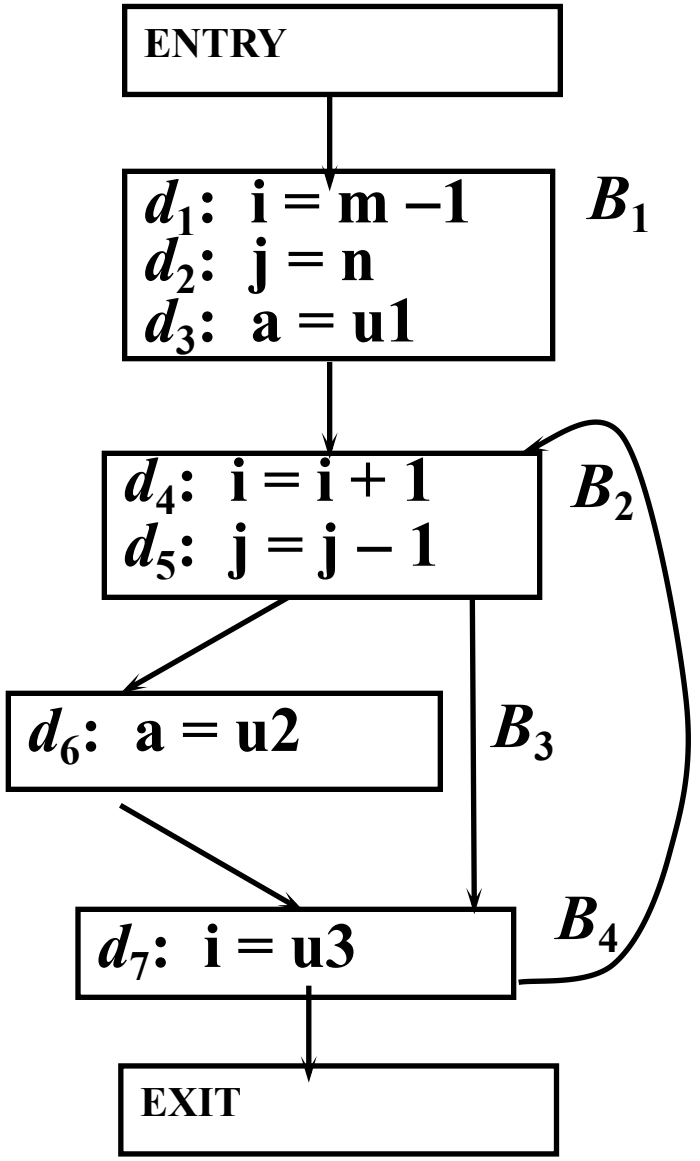
- 活跃变量定义及应用
- **活跃变量分析算法**
- **示例驱动的分析流程**

活跃变量分析

- **相关定义**
  - **IN[*B*]**：块***B***开始点的活跃变量集合
  - **OUT[*B*]**：块***B***结束点的活跃变量集合
  - use_B：块***B***中有引用，且**在引用前在B中没有被定值**的变量集
  - def_B：块B中有定值，且该定值前在B中没有被引用的变量集

use与def的计算

- 例
  - $use[B_1] = \{ m, n, u1 \}$
  - $def[B_1] = \{i, j, a\}$
  - $\boldsymbol{use[B_2] = \{ i, j \}}$
  - $\boldsymbol{def[B_2] = \{\}}$
  - $use[B_3] = \{ u2\}$
  - $def[B_3] = \{a\}$
  - $use[B_4] = \{ u3 \}$
  - $def[B_4] = \{i\}$



求解方程组

- **活跃变量分析逆向数据流等式**
  - IN [EXIT] = ∅
    - 边界条件：程序出口处没有活跃变量
  - **OUT[*B*] = ∪ $_{S是B的后继}$ IN [*S*]**
  - **IN [*B*] = $use_B \cup$ (OUT [*B*] – def_B)**
    - 入口处活跃：1）在B中重定值之前被使用；2)离开时活跃且没有在B中被定值
- **和到达- 定值等式之间的联系与区别**
  - 都以集合并算符作为它们的汇合算符
  - 信息流动方向相反，IN和OUT的作用相互交换
  - *use*和*def*分别取代*gen*和*kill*
  - 仍然需要最小解

活跃变量的迭代计算算法

输入：流图G，其中每个基本块B的use和def都已计算

输出：IN[B]和OUT[B]

IN[EXIT] = Ø;

for (除了EXIT以外的每个块B) IN[B] = Ø;

while (某个IN值出现变化) {

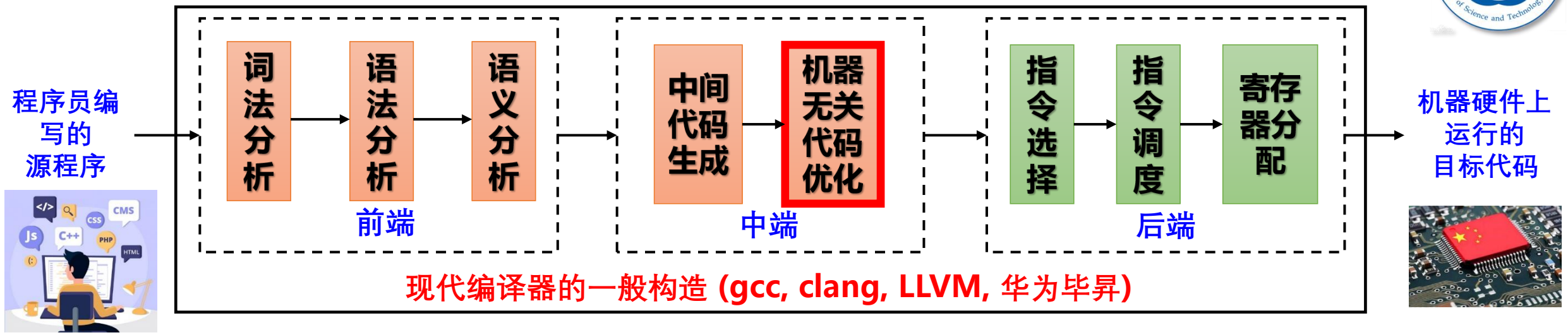
for (除了EXIT以外的每个块B) {

$$\mathrm{OUT}[B] = \cup_{S\text{是}B\text{的后继}} \mathrm{IN}[S]$$

$$\mathrm{IN}[B] = use_B \cup (\mathrm{OUT}[B] - def_B);$$

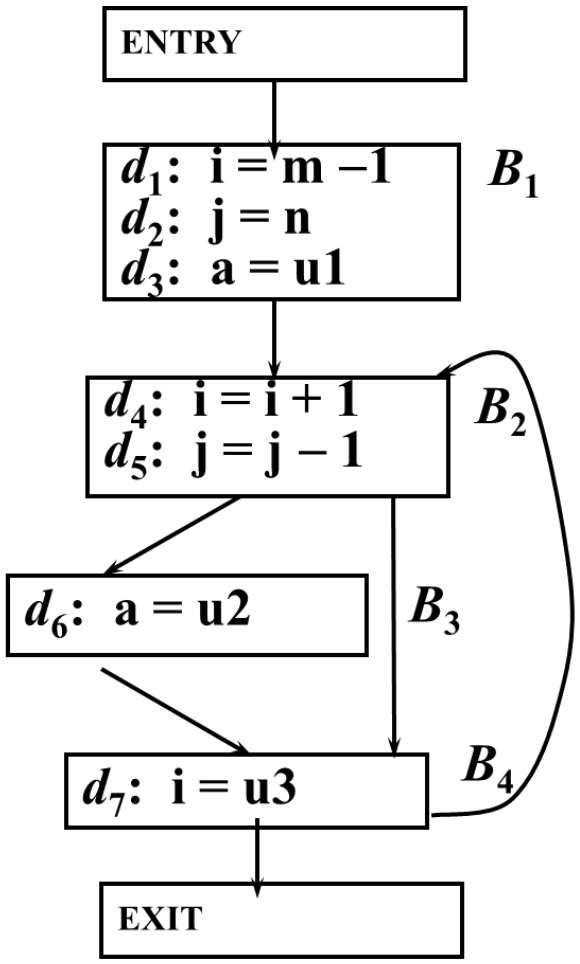
}}

本节提纲



- 活跃变量定义及应用
- **活跃变量分析算法**
- **示例驱动的分析流程**

活跃变量分析-举例1



$use[B_1]$ = { m, n, u1 }
$def[B_1]$ = {i, j, a}
$use[B_2]$ = { i, j }
$def[B_2]$ = {}
$use[B_3]$ = { u2}
$def[B_3]$ = {a}
$use[B_4]$ = { u3 }
$def[B_4]$ = {i}

IN[EXIT] = Ø;

for (除了EXIT以外的每个块B) IN[B] = Ø;

while (某个IN值出现变化) {

for (除了EXIT以外的每个块B) {

$OUT[B] = \cup_{S是B的后继} IN[S]$

$IN[B] = use_B \cup (OUT[B] - def_B)$;

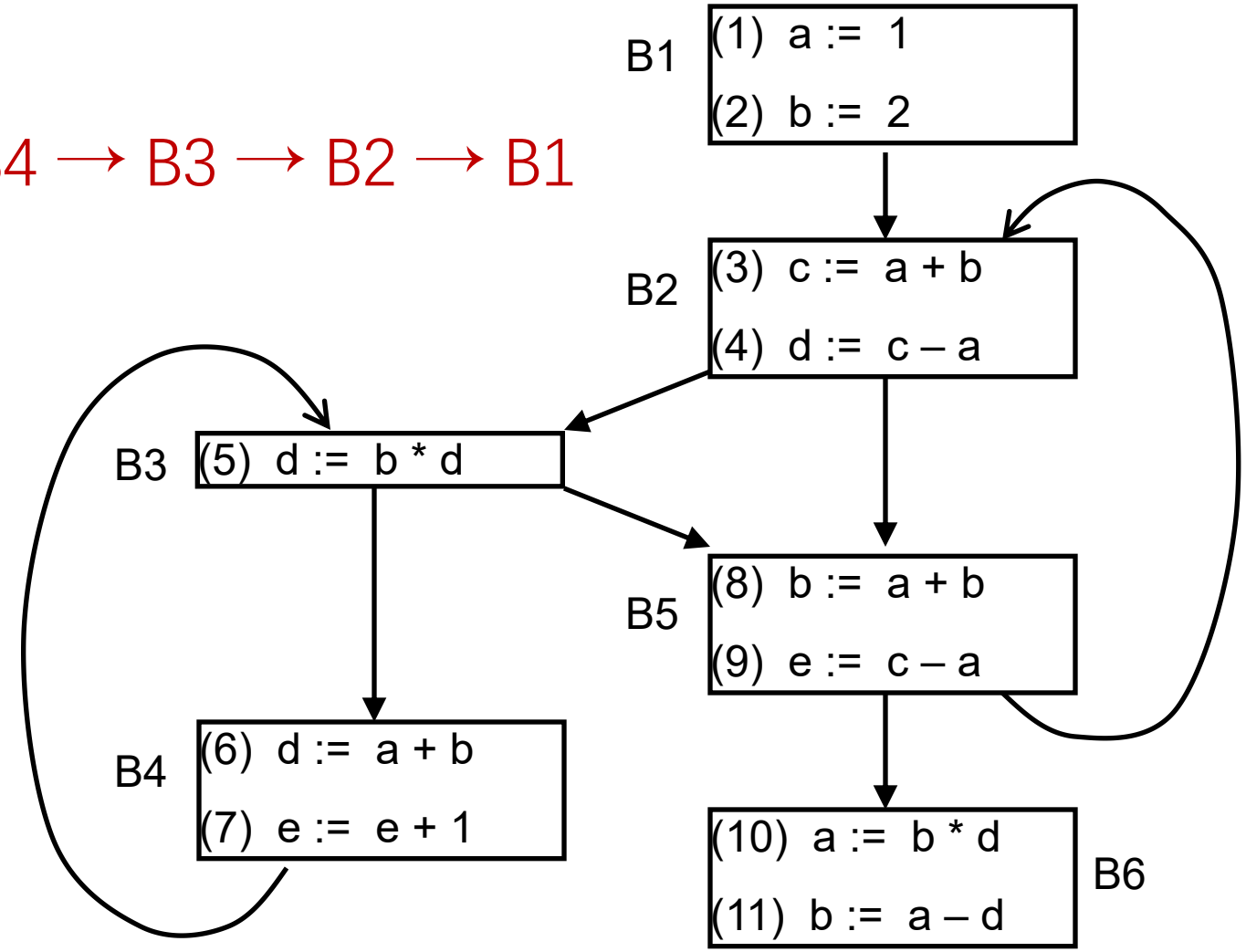
}}

	$OUT[B]^1$	$IN[B]^1$	$OUT[B]^2$	$IN[B]^2$	$OUT[B]^3$	$IN[B]^3$
B_4		u3	i, j, u2, u3	j, u2, u3	i, j, u2, u3	j, u2, u3
B_3	u3	u2, u3	j, u2, u3	j, u2, u3	j, u2, u3	j, u2, u3
B_2	u2, u3	i, j, u2, u3	j, u2, u3	i, j, u2, u3	j, u2, u3	i, j, u2, u3
B_1	i, j, u2, u3	m, n, u1, u2, u3	i, j, u2, u3	m, n, u1, u2, u3	i, j, u2, u3	m, n, u1, u2, u3

活跃变量分析-举例2

计算次序

* B6 → B5 → B4 → B3 → B2 → B1



基本块出口活跃变量

- 各基本块USE和DEF如下，

  USE[B1] = { } ; DEF[B1] = { a, b }

  USE[B2] = { a, b } ; DEF[B2] = { c, d }

  USE[B3] = { b, d } ; DEF[B3] = { }

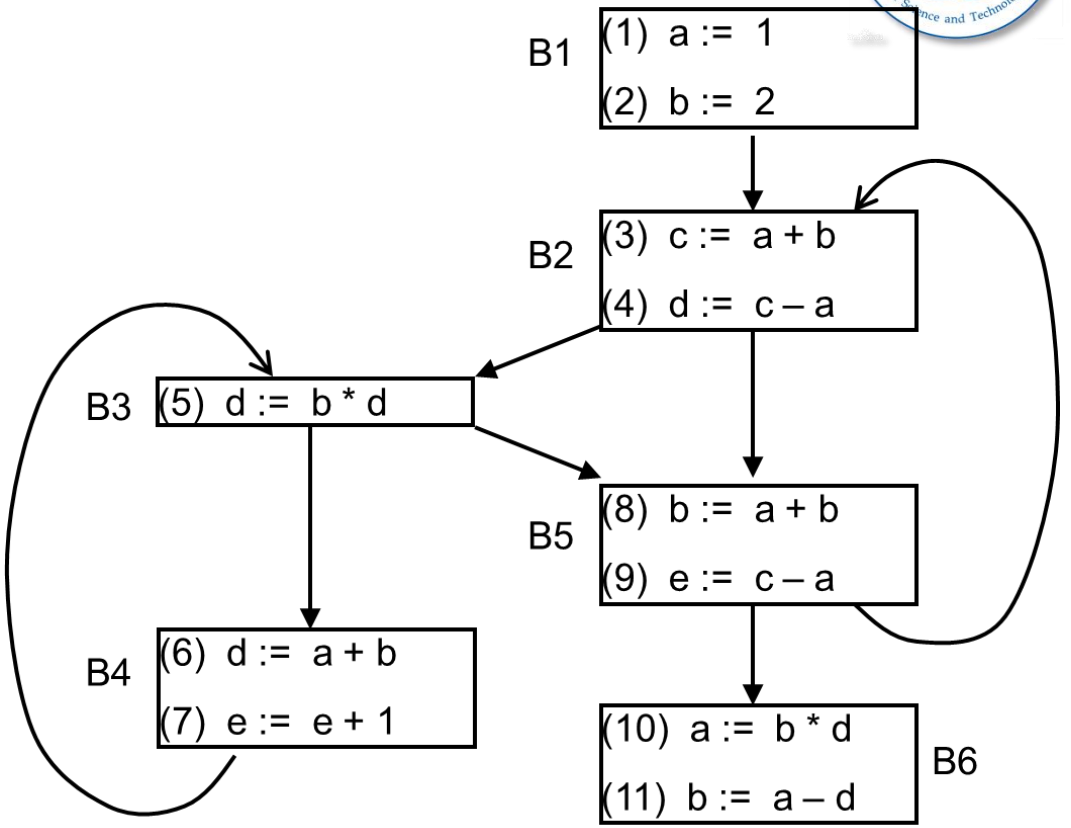
  USE[B4] = { a, b, e } ; DEF[B4] = { d }

  USE[B5] = { a, b, c } ; DEF[B5] = { e }

  USE[B6] = { b, d } ; DEF[B6] = { a }

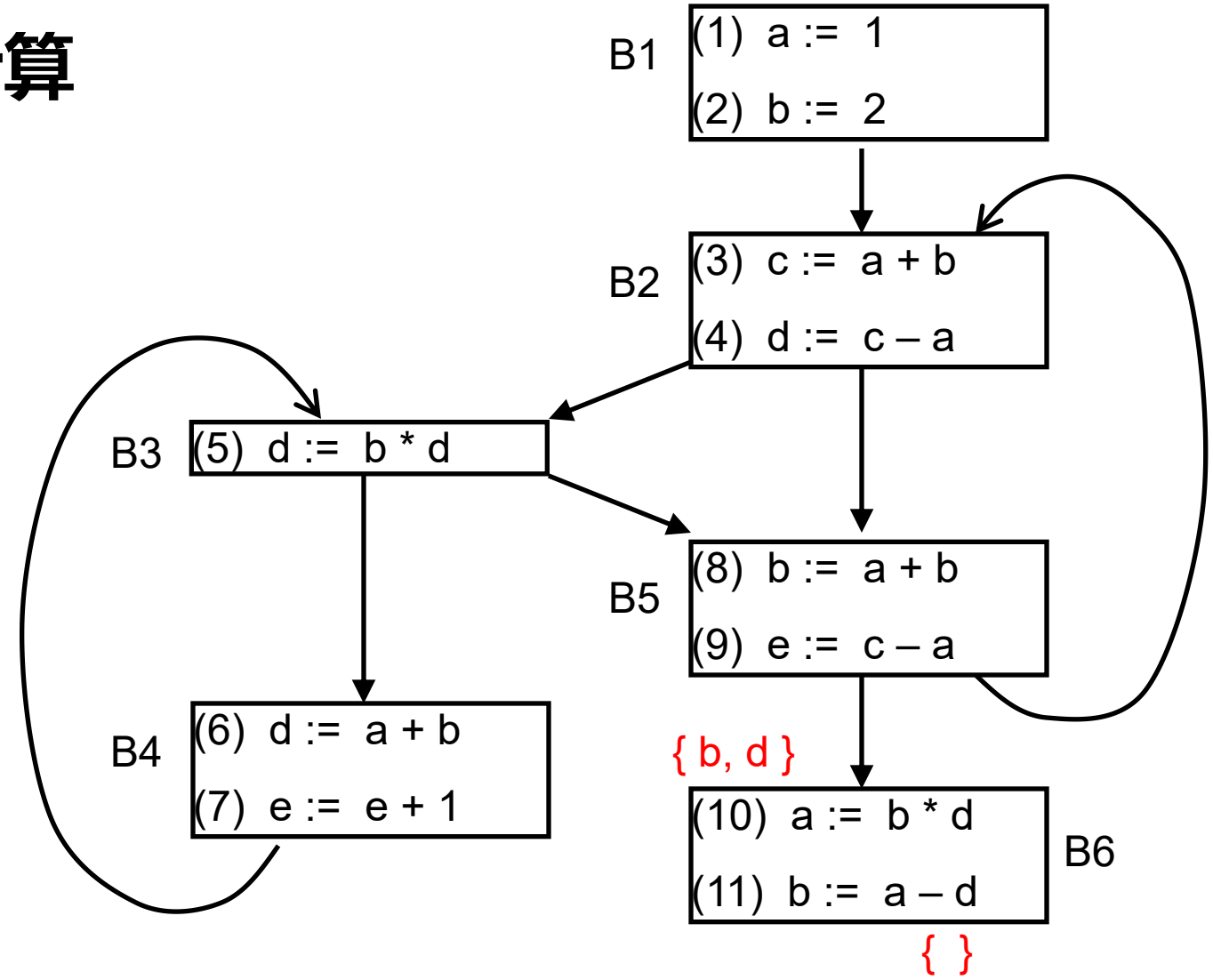
- 初始值，all B, IN[B] = { },

  OUT[B6]={ }//出口块



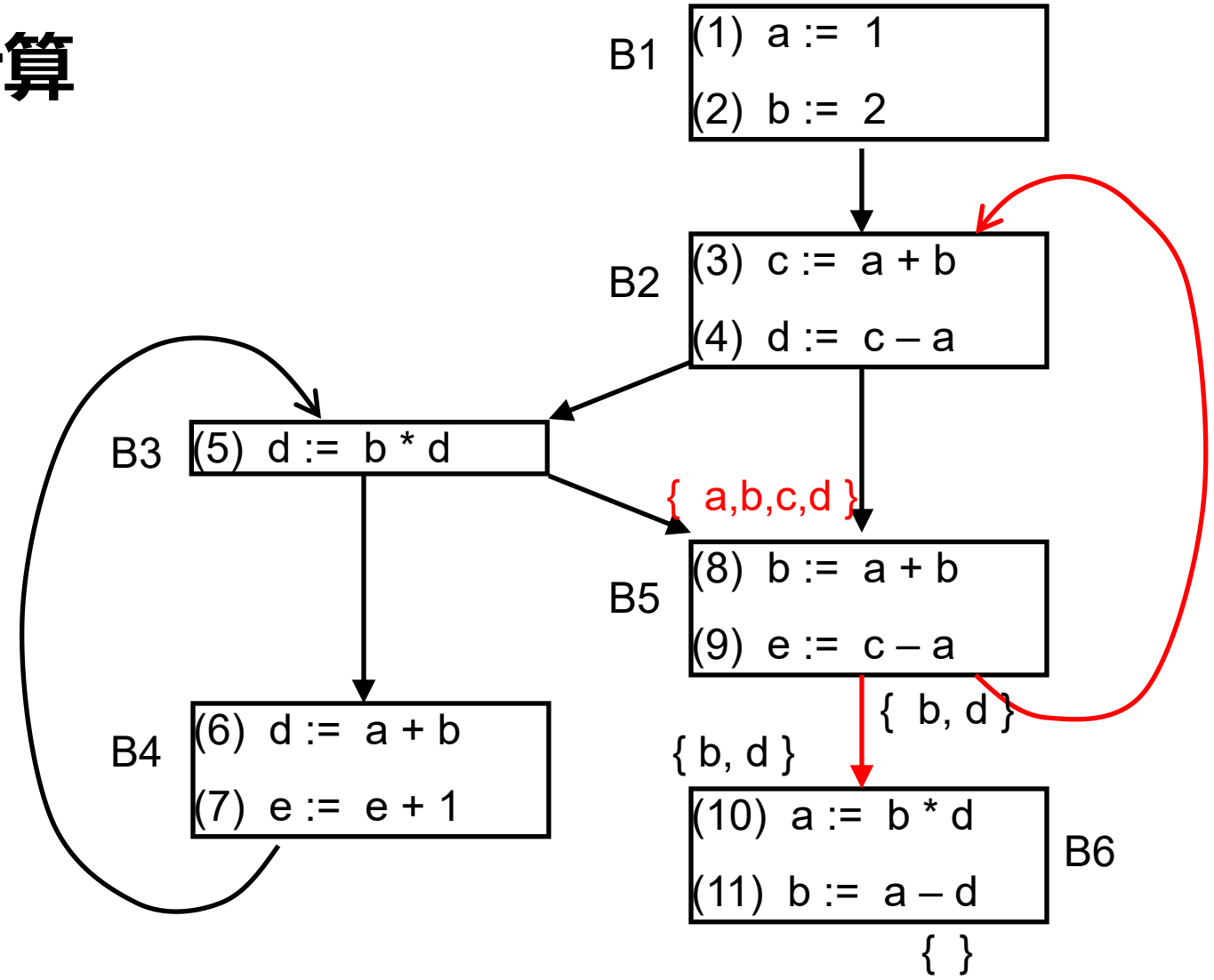
基本块出口活跃变量

- 第一次迭代计算



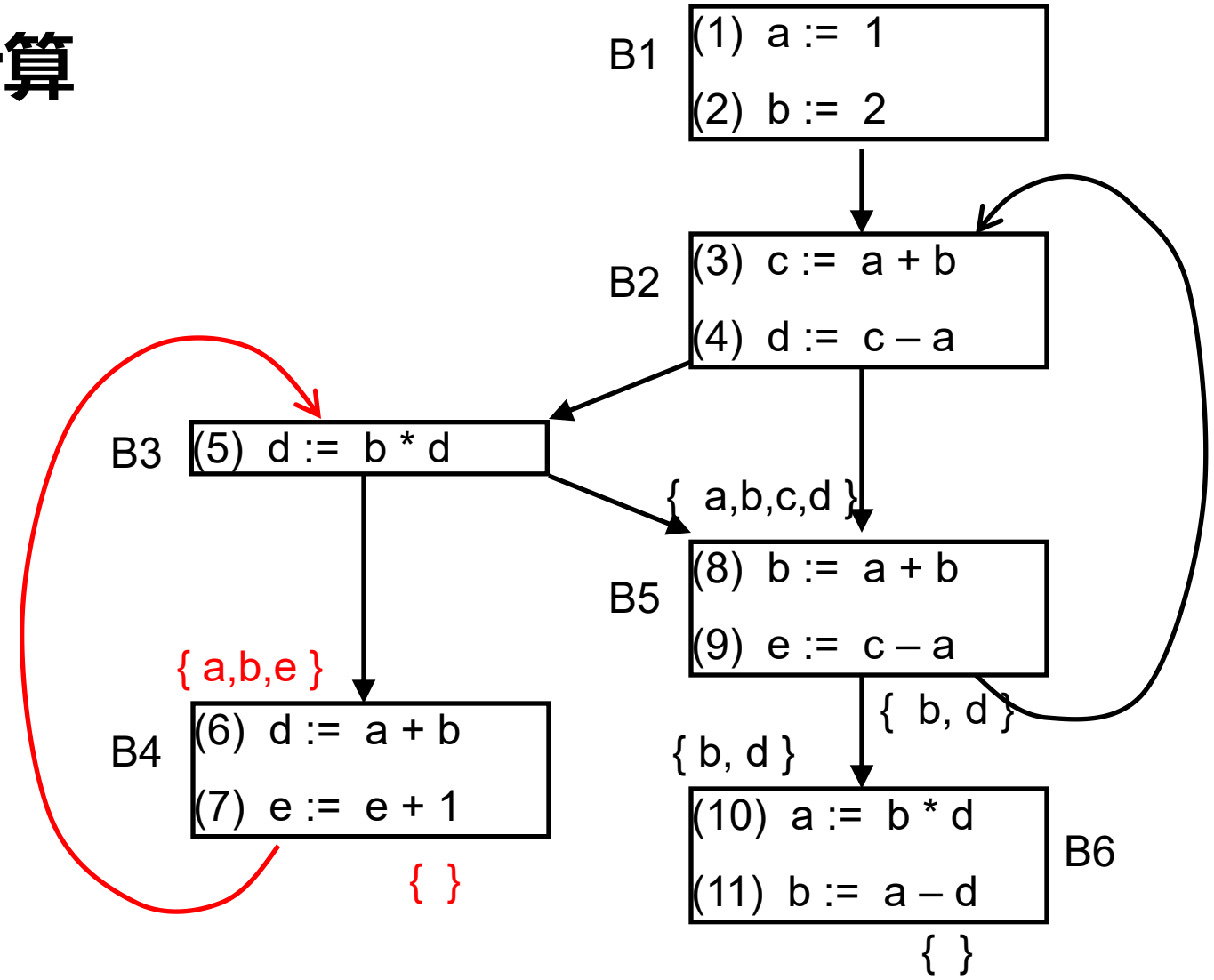
基本块出口活跃变量

- 第一次迭代计算



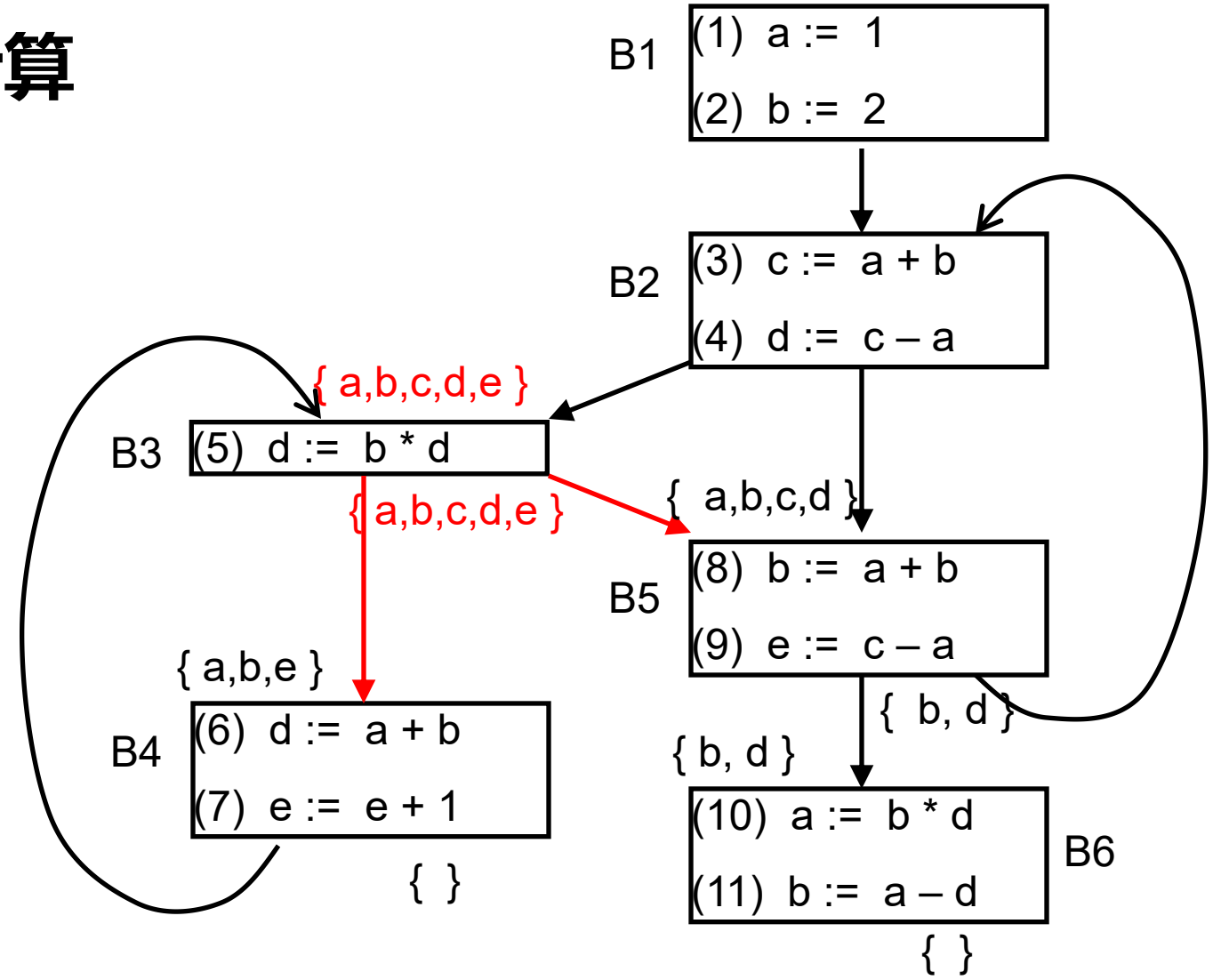
基本块出口活跃变量

- 第一次迭代计算



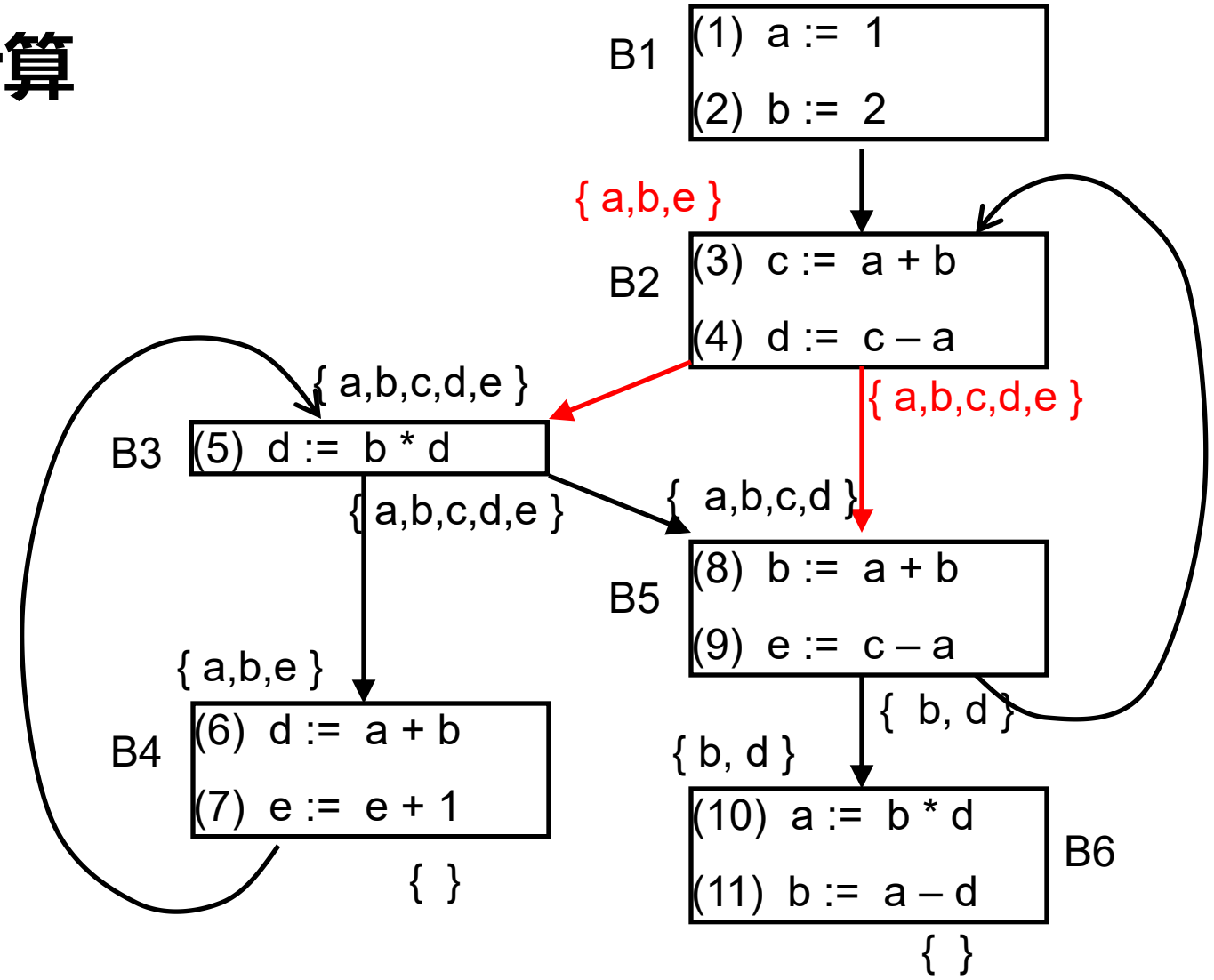
基本块出口活跃变量

- 第一次迭代计算



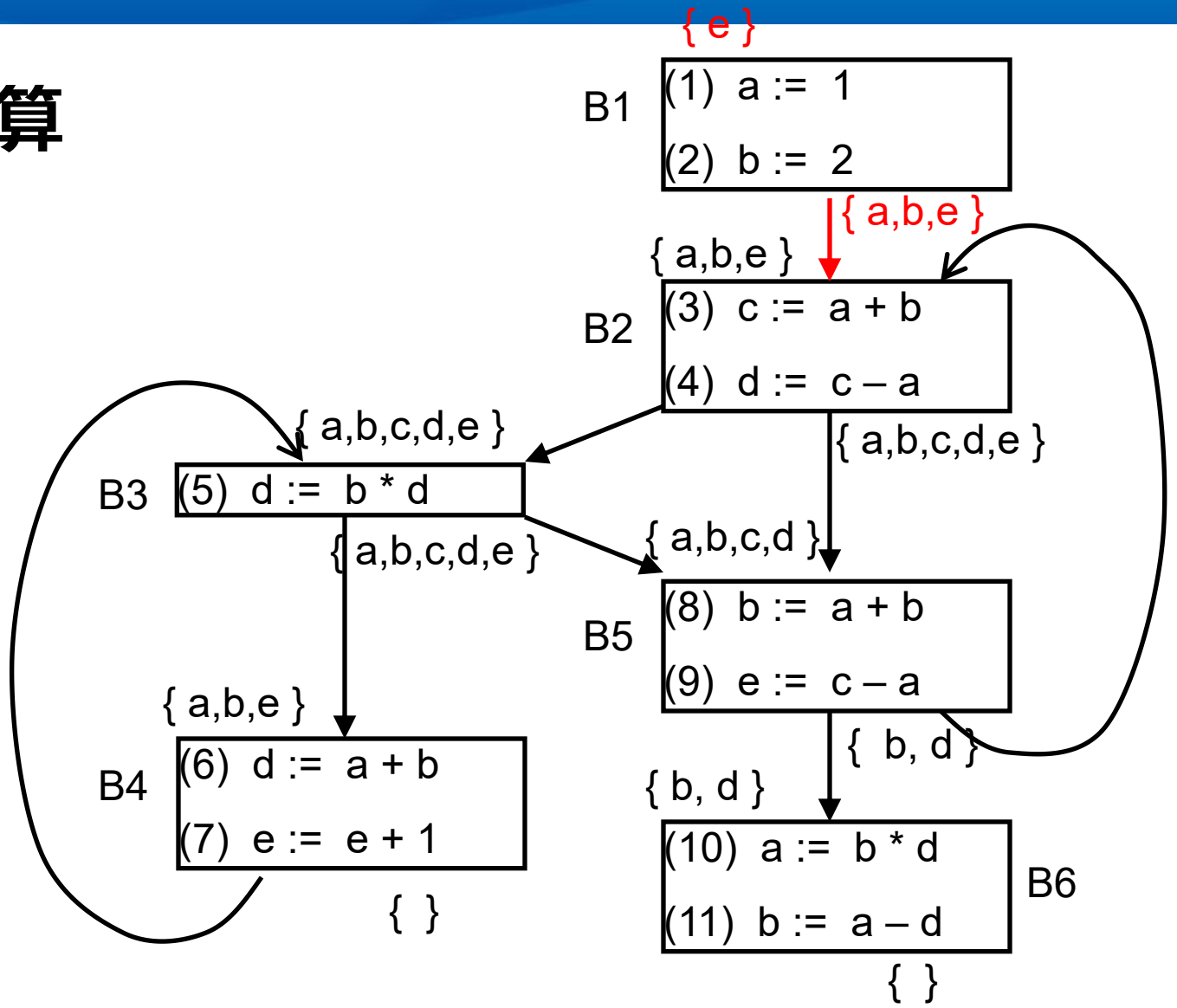
基本块出口活跃变量

- 第一次迭代计算



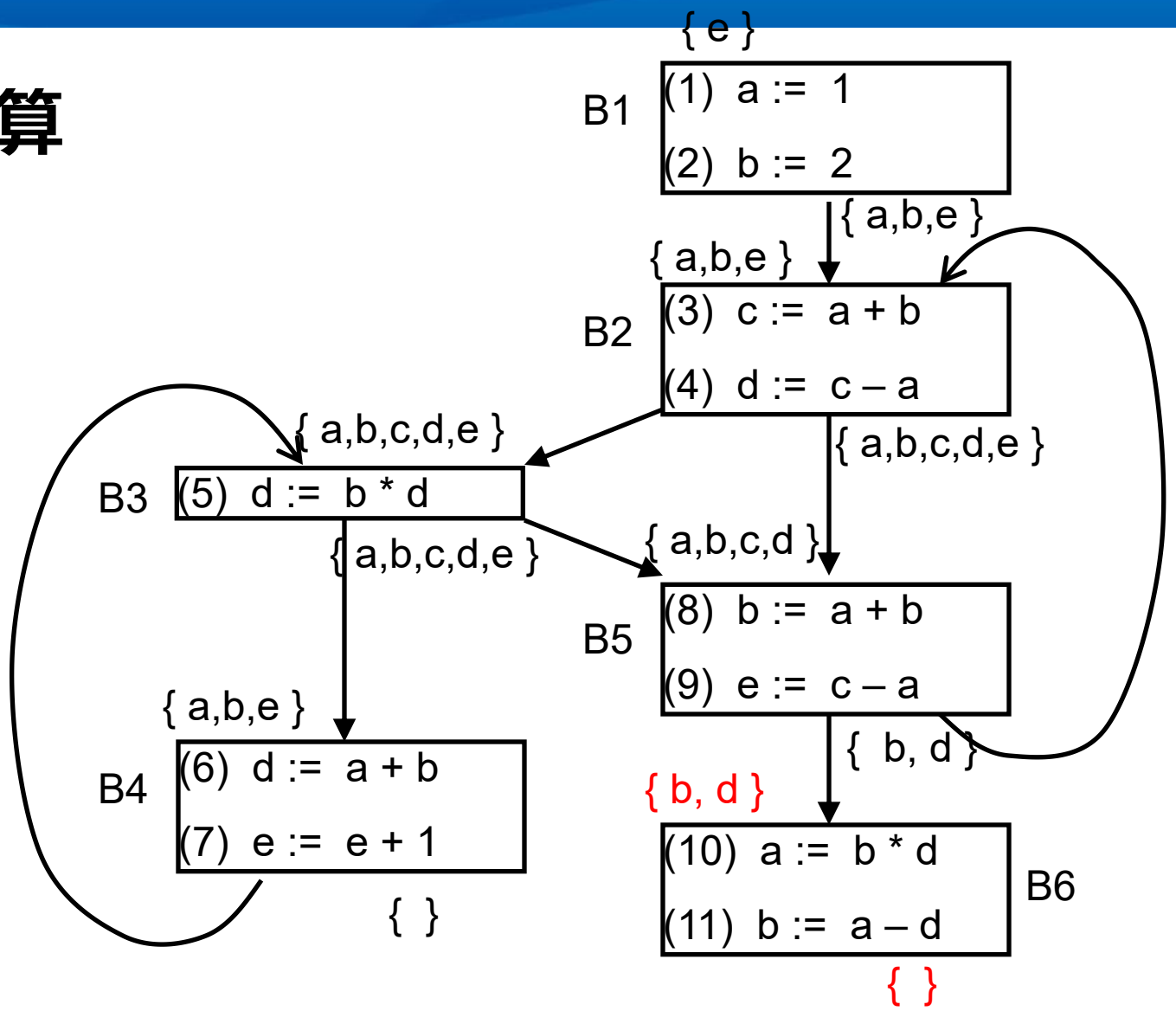
基本块出口活跃变量

- 第一次迭代计算



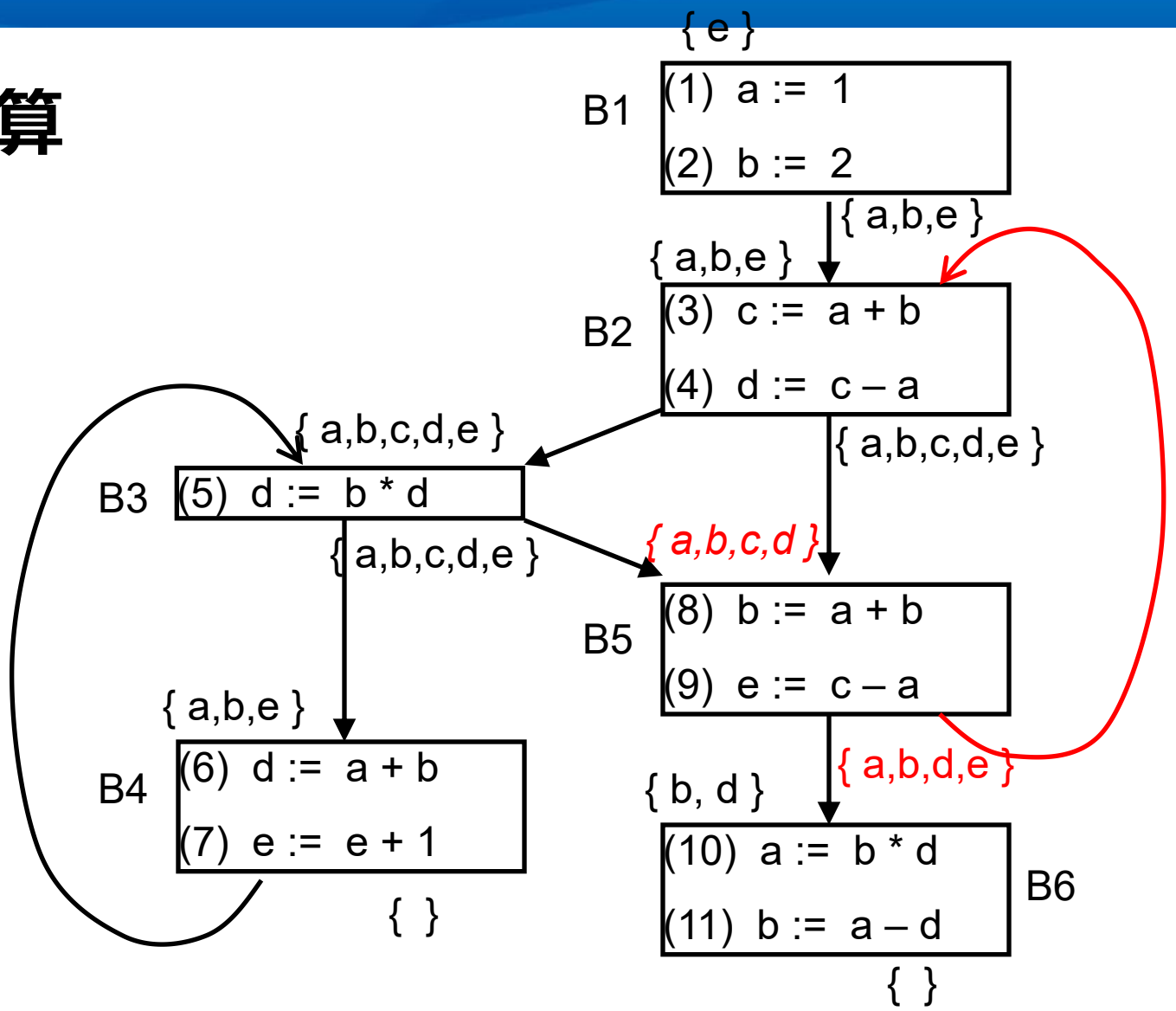
基本块出口活跃变量

- 第二次迭代计算



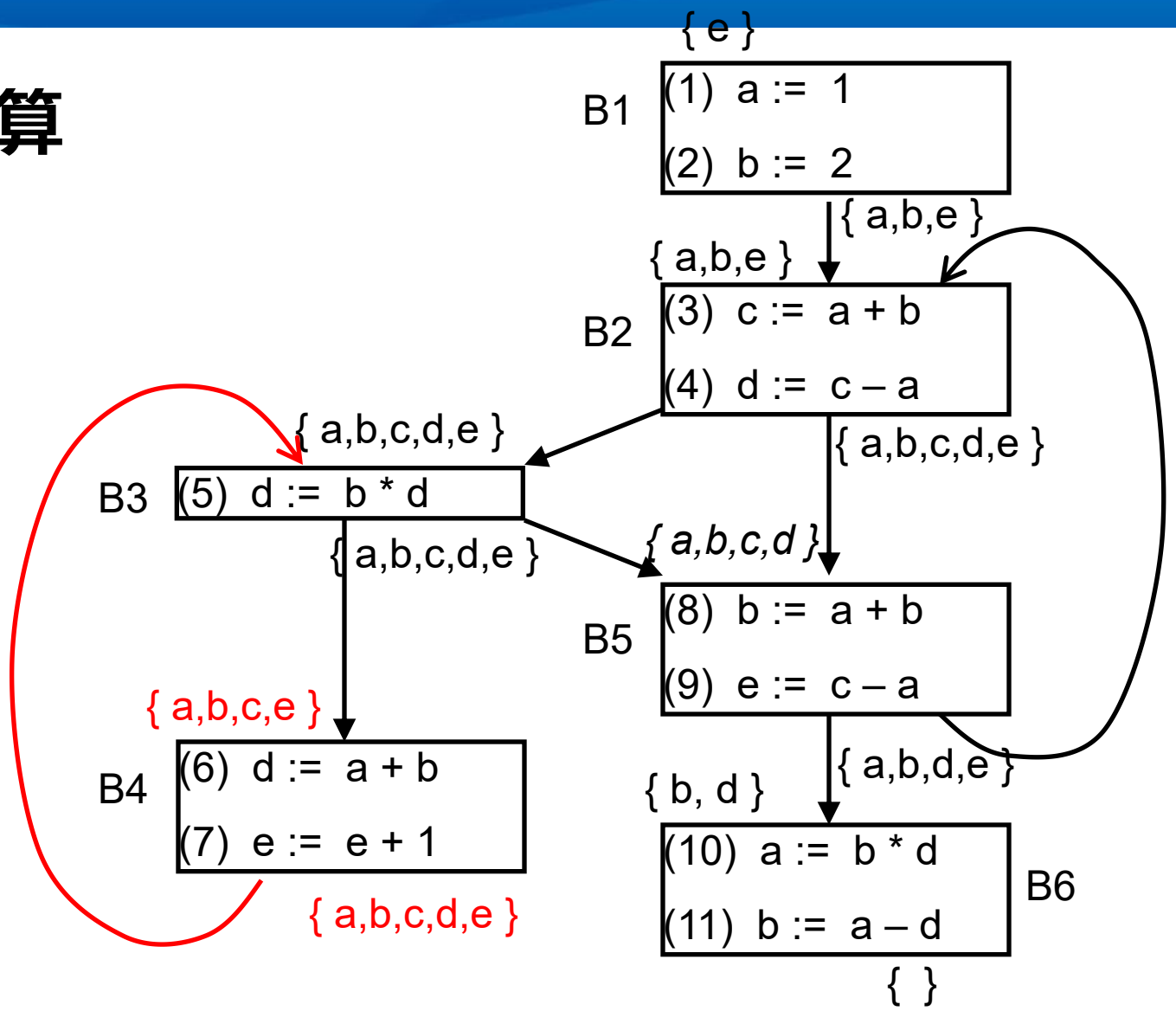
基本块出口活跃变量

- 第二次迭代计算



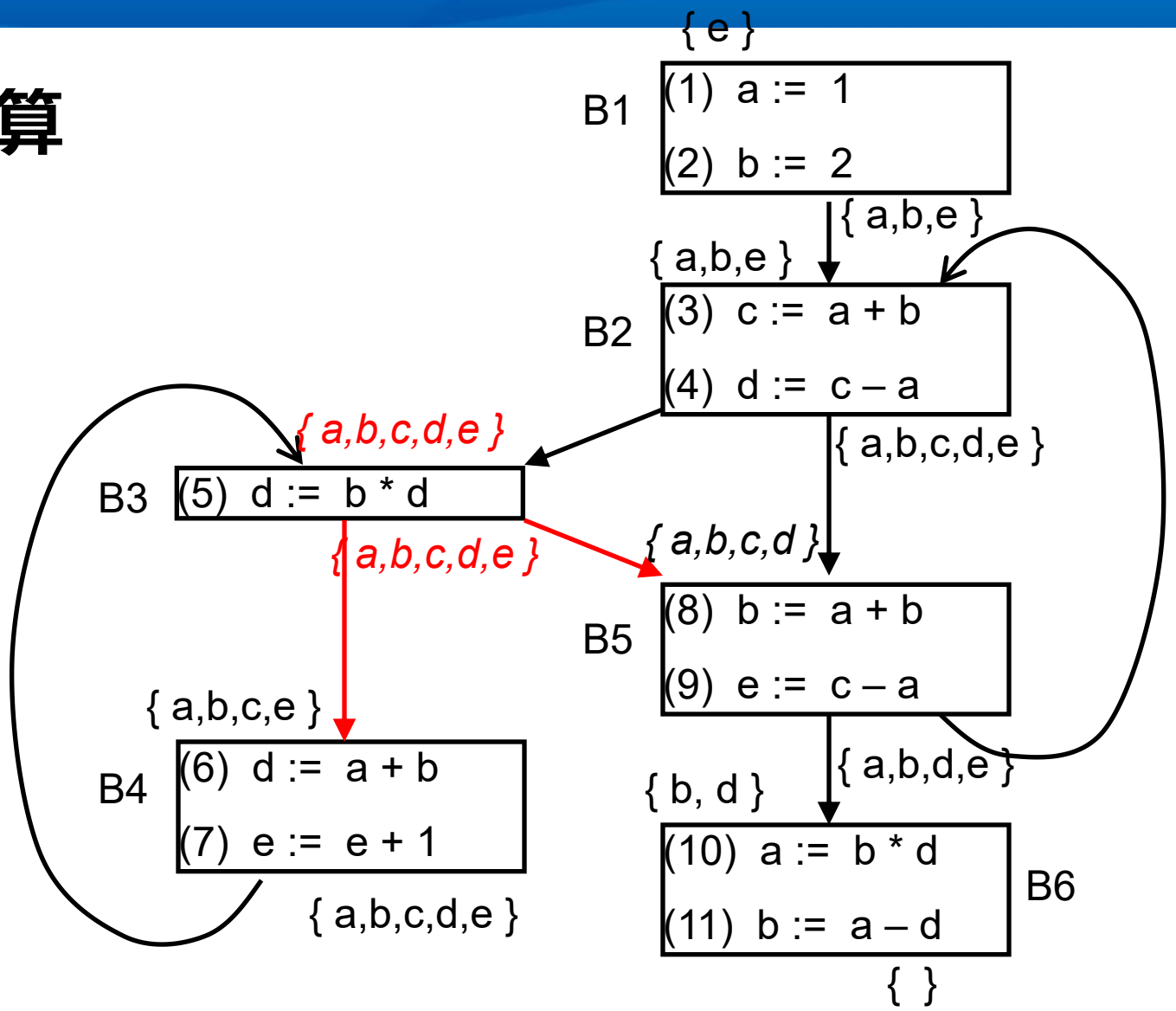
基本块出口活跃变量

- 第二次迭代计算



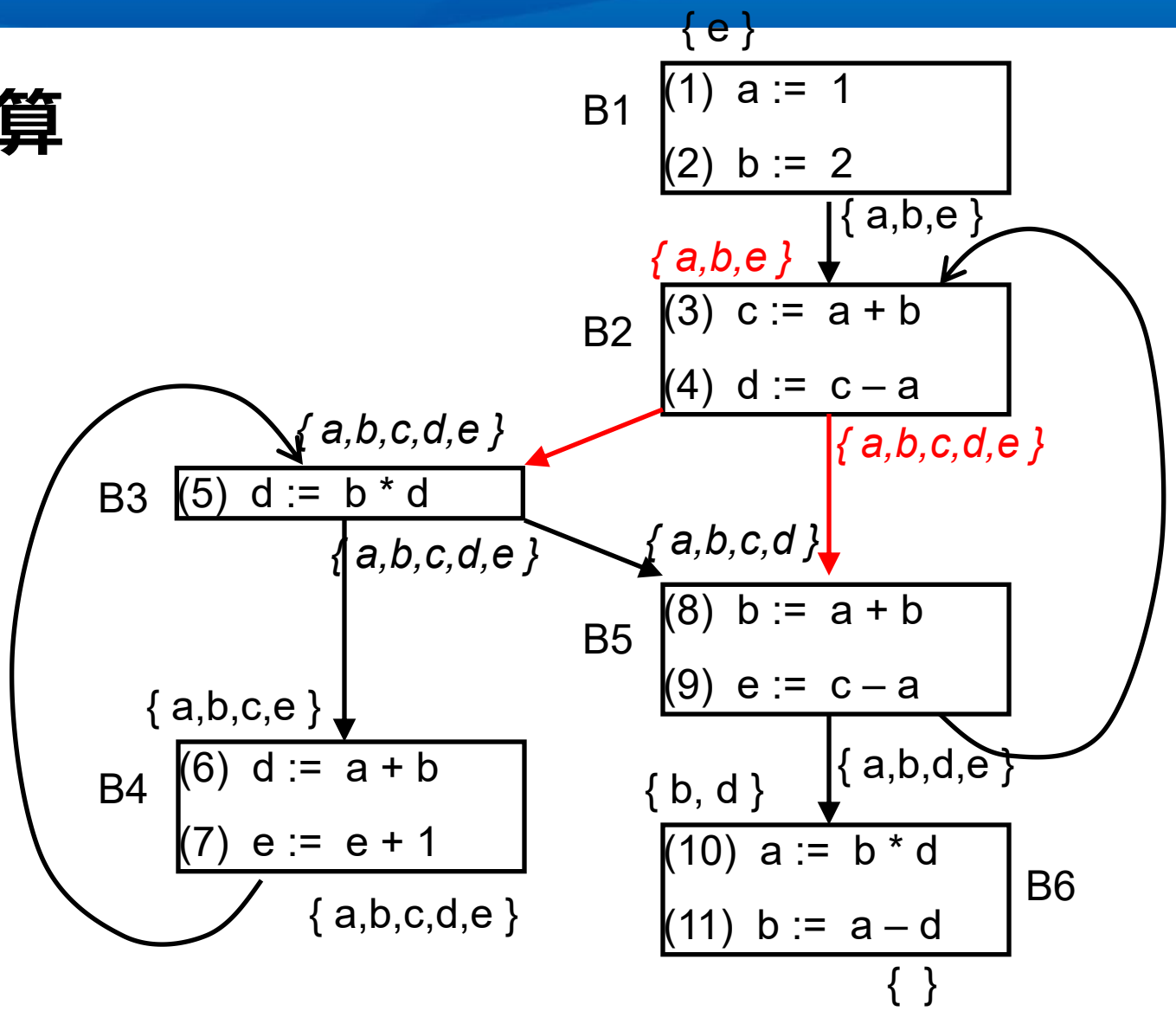
基本块出口活跃变量

- 第二次迭代计算



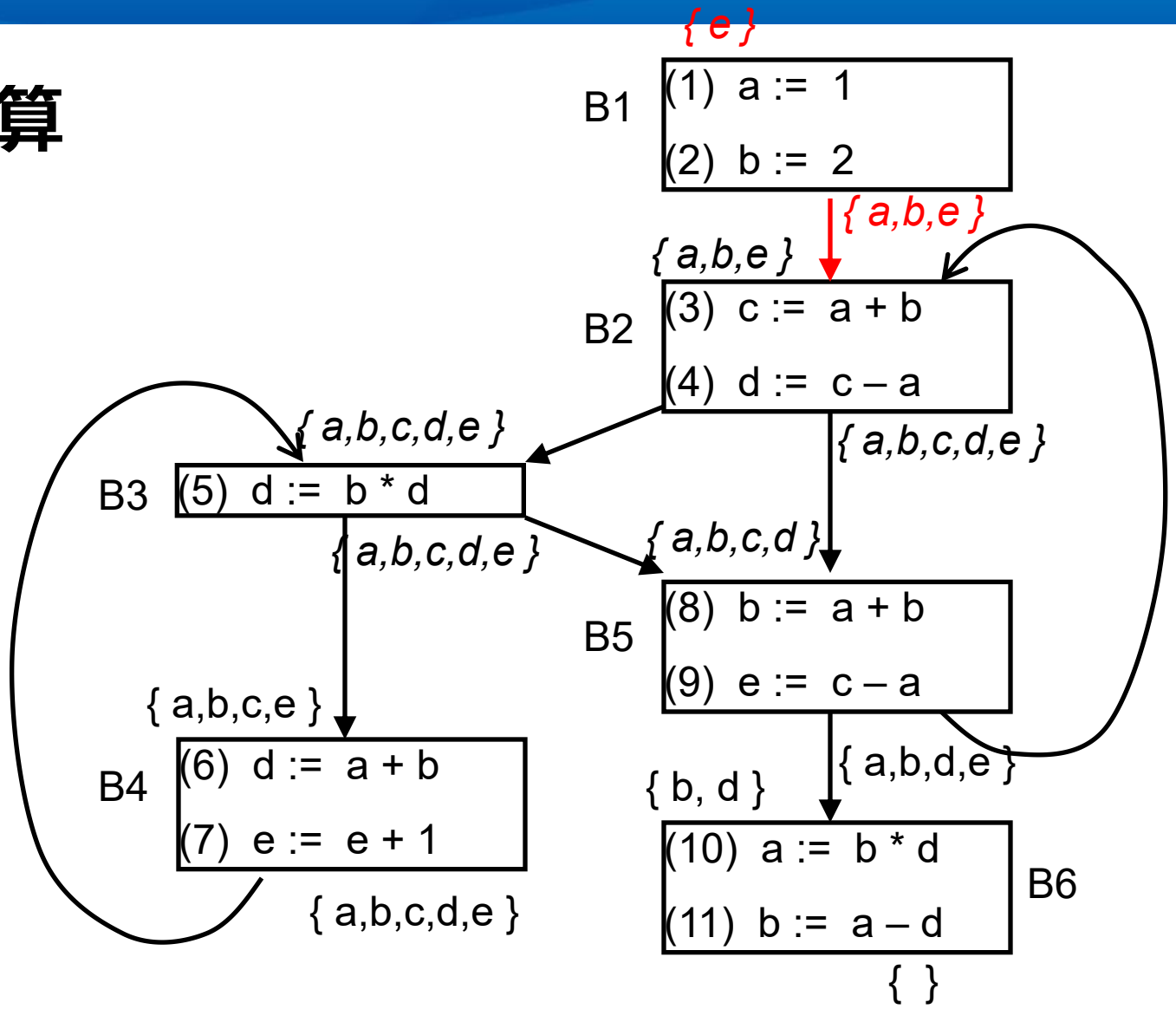
基本块出口活跃变量

- 第二次迭代计算



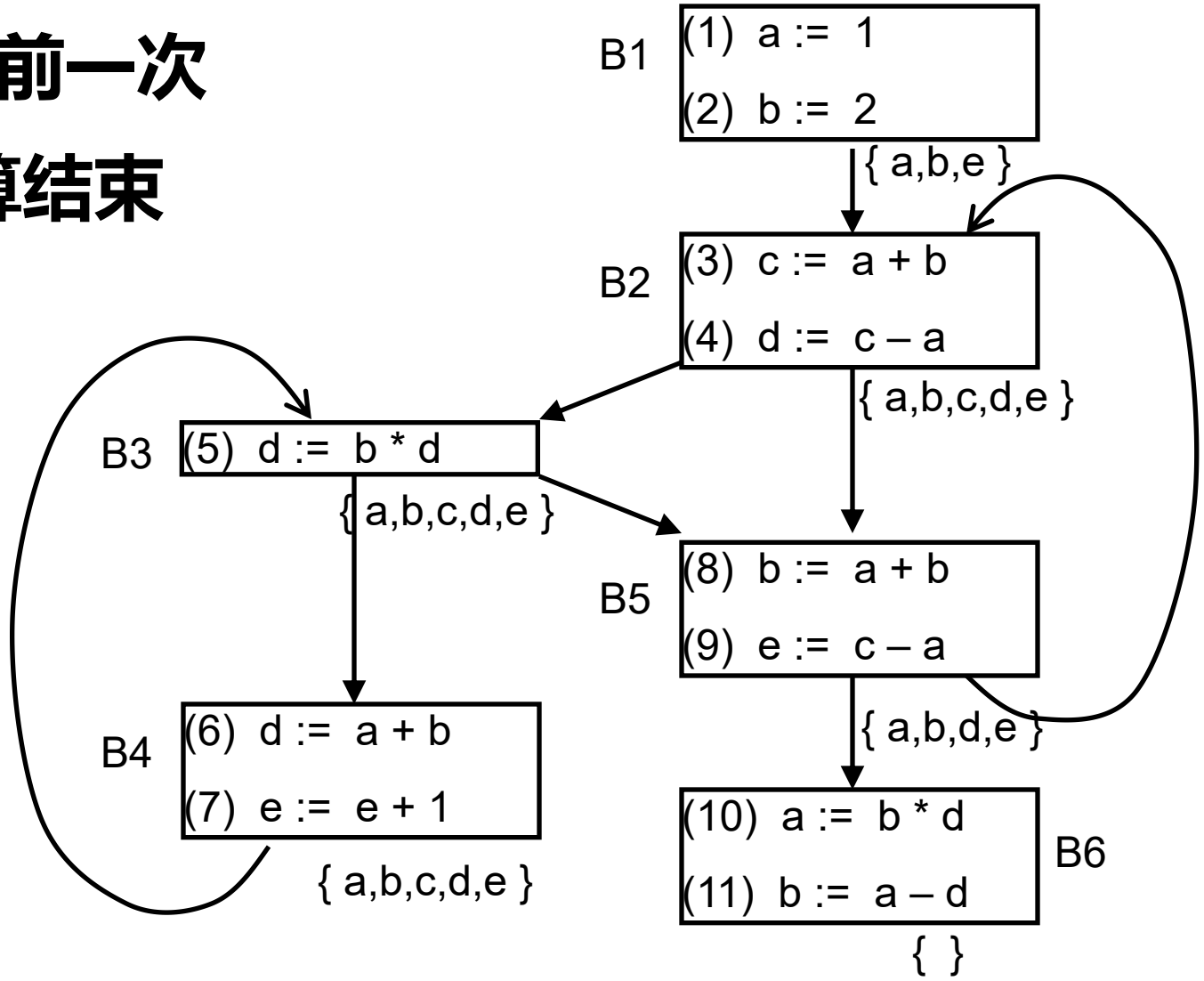
基本块出口活跃变量

- 第二次迭代计算



基本块出口活跃变量

- 第三次迭代与前一次结果一样，计算结束



教会AI写代码，但由你定义灵魂！

李 诚

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2026年11月9日